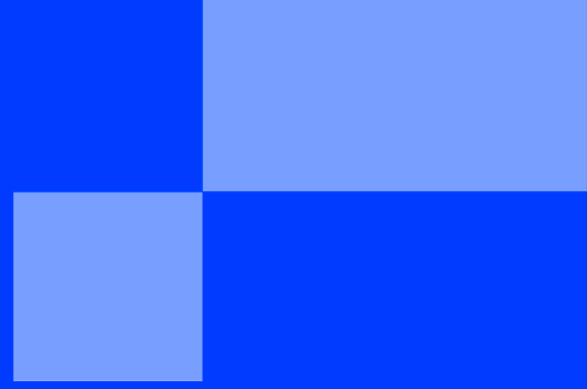




CARTRIDGE CUSTOM

< UN SUJET CUSTOM POUR LES MALADES />

CARTRIDGE CUSTOM

Dans le cadre du projet Cartridge proposé par Epitech, j'ai décidé d'écrire un nouveau sujet parallèle pour les groupes qui le souhaitent afin de plonger plus intensément dans la programmation dite "bas-niveau" ou encore "bare-metal". Ce sujet, qui est un mix entre un gros bootstrap et un sujet, tentera de se positionner comme une porte d'entrée au domaine de la programmation matérielle en couvrant principalement les thèmes suivants :

- ✓ la conception d'outils spécialisés
- ✓ l'utilisation de compilateur spécialisé (cross-compileur)
- ✓ la conception de driver
- ✓ algorithmie de dessin (dessiner des lignes, des caractères, des cercles, ...)
- ✓ un peu de rétro-ingénierie afin de comprendre le comportement de la machine
- ✓ l'assembleur ARM

Néanmoins, l'une des grosses différences avec le sujet original (au-delà du fond), c'est que l'on va migrer sur la [GameBoy Advance](#) plutôt que la [GameBoy](#) "classique". La raison principale, c'est qu'on passe d'un processeur z80 à un processeur ARM qui est infiniment plus d'actualité aujourd'hui (contrairement au z80 qui n'existe pratiquement plus (modulo certaines calculatrices de chez [Texas Instrument](#))) et j'ai envie de vous faire apprendre quelque chose qui résonne à minima avec le présent.

Ce sujet est divisé en plusieurs "parties" / "chapitres" progressifs avec des parutions fréquentes (le sujet ne sera pas disponible d'une traite). Déjà pour me laisser le temps d'écrire, et pour vous, qui tentez de faire ce sujet, le temps de digérer les nouvelles notions qui ne sont pas si simples à appréhender lorsque l'on tombe dessus la première fois. Tout naturellement, je vous invite aussi à me faire des retours pour que je puisse modifier et améliorer le sujet avec le temps.

Parlons maintenant du projet, votre but sera de concevoir un outil de diagnostic matériel. L'idée étant de vous permettre d'observer le comportement de la machine pour comprendre les différents mécanismes internes du matériel (timer, DMA, écran, ...) afin de commencer à l'exploiter à son plein avantage. En parallèle, vous allez concevoir un SDK (une grosse boîte à outils) pour vous aider à déboguer et à naviguer dans le projet et qui vous permettra de vous concentrer sur les notions principales.

Bon courage pour le projet !

Partie I - L'écran rouge

Dans cette première partie de sujet, nous allons principalement nous concentrer sur la mise en place de notre environnement de développement avec comme but final de réussir à afficher un écran rouge dans un émulateur spécialement taillé pour ce sujet.

Je sais que ça fait de la peine dit comme ça, mais vous n'imaginez pas encore toute l'ingénierie qu'il y a derrière cette simple première partie du projet juste pour faire quelque chose d'aussi simple.

Il faut bien garder à l'esprit que, contrairement au programme que vous écrivez sur votre machine et qui tourne grâce à votre système d'exploitation, ici, nous sommes **seuls** avec les composants physiques. Ce qui signifie que si vous voulez afficher un pixel à l'écran... il faut aller discuter directement avec l'écran pour lui dire d'afficher votre pixel... si tant est que l'écran est alimenté, parce que si ce n'est pas le cas, il faut d'abord aller discuter avec la batterie pour leur dire d'aller alimenter l'écran... C'est très schématique, mais on n'est pas loin de la réalité.

Avant de se lancer corps perdu dans la programmation, il est super important de mettre en place, en amont, un environnement de développement et de débogage lorsque l'on se lance dans un projet aussi bas-niveau sur une machine inconnue.

Contrairement, encore une fois, à vos programmes "classiques", ici, nous avons beaucoup d'étapes pour pouvoir exécuter notre jeu et la moindre mauvaise instruction peut potentiellement être critique pour le matériel, pouvant l'amener à faire planter physiquement la machine ou pire, la "[bricker](#)" (la casser de manière irrémédiable). Il faut donc bien prendre le temps de mettre en place des outils pour nous aider.

Cette première partie peut donc sembler frustrante à première vue, car nous n'allons pas entrer dans les détails techniques (vous n'allez même rien développer du tout sur la machine, je vous donnerai un bout de code qui fera le travail à votre place), mais elle va nous permettre de poser les fondations solides pour pouvoir naviguer dans votre projet (en plus de vous expliquer les premiers concepts matériels).

todo: image red screen

Trêve de blabla théorique, parlons maintenant de la machine, la fameuse GameBoy Advance (GBA)!

Lorsque l'on désire programmer à très bas niveau (comprenez par là "lorsque l'on cherche à contrôler le matériel"), il est important de savoir d'où est-ce que l'on part en termes purement d'exécution de programme, et est-ce qu'il y a des obstacles qui nous empêcheraient de prendre le contrôle du matériel ?

Pour notre cas, nous savons trois choses :

1. on est seul à s'exécuter sur le matériel
2. notre code se trouve dans la cartouche de jeu
3. la cartouche semble être exécutée directement au démarrage

Donc, méthodologiquement, la première chose que l'on va essayer de comprendre, c'est la façon dont est structurée une cartouche de la GameBoy Advance.

Cependant... pourquoi une cartouche et pas un autre format ? Une notion super importante à garder à l'esprit lorsque l'on investit une machine pour la première fois, c'est le contexte économique et historique autour de la console. Du moins, essayer de comprendre ce qui a poussé le constructeur à prendre certaines décisions techniques à ce moment-là de l'histoire.

Dans notre cas, à l'époque de la sortie de la machine (et encore aujourd'hui!), les constructeurs de consoles vendent leurs produits à perte (ou presque)! Le but de l'opération n'est pas de vendre le plus de consoles pour engranger le plus d'argent, mais simplement d'avoir le plus de personnes qui achètent la console. Pourquoi ? Parce que les bénéfices, eux, sont pratiquement exclusivement engendrés par les jeux. Tant par le prix des licences que les studios de développement paient à Nintendo pour pouvoir créer leurs jeux, que le pourcentage du prix taxé sur chaque cartouche vendue.

Le constat est donc clair : que ce soit pour la GBA ou toute autre console, les jeux doivent être suffisamment sécurisés pour que personne ne puisse concevoir / vendre des produits sur une console sans demander directement au constructeur directement (dans notre cas Nintendo).



C'est d'ailleurs cette position ferme et ce droit de regard invasif sur les studios de développement (car Nintendo a refusé des projets à beaucoup de studios) qui leur ont permis de maintenir une certaine exigence de qualité sur ce qu'ils sortaient ou non sur leur matériel. Cette approche stricte leur a permis de faire remonter la confiance des consommateurs après le crash (américain) du jeu vidéo causé par Atari en 1980 et a permis façonner le monde vidéoludique que l'on connaît aujourd'hui.

Il y a donc une ombre au tableau : la cartouche ne peut pas simplement contenir le code du jeu, sinon tout le monde aurait créé leur propre jeu dessus, il y a donc une arnaque quelque part.

Pour confirmer nos suspicions, nous pouvons simplement prendre une ROM (qui est simplement une copie logicielle (un "dump") d'une cartouche GBA) et de simplement essayer d'afficher les instructions pour

tenter de voir si nous avons du code qui ressemble à quelque chose. Si ce n'est pas le cas, c'est qu'il y a effectivement une protection de sécurité mise en place par Nintendo pour éviter de trop facilement concevoir des jeux sur la machine.

Voici l'affichage (fortement simplifié) du début de la cartouche de [Super Mario Advance 2](#):

ADDR	CODE	INSTRUCTION
0:	ea00002e	b 0xc0
4:	51aeff24	??
8:	21a29a69	??
c:	0a82843d	??
10:	ad09e484	??
14:	988b2411	??
18:	217f81c0	??
1c:	19be52a3	??
20:	20ce0993	??
24:	4a4a4610	??
28:	ec3127f8	??
2c:	33e8c758	??
30:	bfcce382	??
34:	94dff485	??
...		
c0:	e3a00012	mov r0, #18
c4:	e129f000	msr CPSR_fc, r0
c8:	e59fd028	ldr sp, [pc, #40] @ 0xf8
cc:	e3a0001f	mov r0, #31
d0:	e129f000	msr CPSR_fc, r0
d4:	e59fd018	ldr sp, [pc, #24] @ 0xf4
d8:	e59f1198	ldr r1, [pc, #408] @ 0x278
dc:	e28f0018	add r0, pc, #24
e0:	e5810000	str r0, [r1]
e4:	e59f1190	ldr r1, [pc, #400] @ 0x27c
...		

Du coup, à part la première instruction qui ressemble à quelque chose de valide, tout le reste ne ressemble à rien... Puis, plus tard (vers 0xc0), on retrouve des instructions qui semblent cohérentes. Il y a donc bien quelque chose de particulier mis en place dans les cartouches, très probablement en lien avec la sécurité de la console. Si on regarde plus en détail la structure d'une cartouche de GBA (qui ont largement été documentées au cours des 25 dernières années), nous pouvons trouver ceci:

todo: schemas format d'une cartouche

Il y a donc bien un en-tête au début de chaque cartouche. Ce qui implique que notre dernière supposition comme quoi "la cartouche semble être exécutée directement au démarrage" est fausse : il y a bien un programme (le BIOS) qui est exécuté en premier sur la machine, qui va s'occuper de checker que la cartouche contient bien un en-tête particulier.

Notre objectif est donc simple : nous devons comprendre comment est structuré le header afin de le répliquer dans le but de nous faire passer aux yeux de la console comme un jeu légitime. Et c'est justement le premier exercice que nous allons faire.



Pour votre culture générale, je vous recommande chaudement la vidéo [Comment un "docteur" a brisé la sécurité de la Game Boy & GB Advance](#) qui retrace l'histoire du hack de la GameBoy et GameBoy Advance

Commençons enfin à programmer quelque chose.

Comme expliqué plus haut, nous devons comprendre comment fonctionne le header d'une cartouche pour pouvoir la répliquer et nous faire passer pour un jeu légitime aux yeux de la console. Ce qui nous donne une bonne excuse pour concevoir notre premier outil qui devra récupérer le header d'une ROM et y afficher les informations.

Cependant, nous devons prendre en compte le fait que nous allons avoir, à la fin du sujet, plusieurs autres outils à concevoir. Certains d'entre eux reposeront sur d'autres de nos propres outils ou seront carrément interdépendants. Une notion qui revient souvent lorsque l'on commence à avoir beaucoup d'outils est le concept de SDK.

Un [Software Development Kit](#) (SDK), est (généralement) simplement un regroupement d'outils permettant d'avoir, clef en main, un environnement de développement complet pour un domaine ou un type de projet particulier. C'est exactement ce que nous allons tenter de réaliser tout au long de ce sujet.

Bien sûr, nous allons commencer par de simples commandes de visualisation, mais vous allez vite vous rendre compte qu'on va avoir besoin d'un outil pour compiler, puis un autre pour lancer l'émulateur, puis un autre pour convertir des images, ... Il faut penser "environnement de développement", sinon vous allez vous parasiter l'esprit à retenir tout un tas de commandes et d'invocations de scripts qui vous empêcheront de vous concentrer sur le principal à savoir : la programmation matérielle.

Nous allons donc commencer par la conception de l'outil "principal" [cartridge](#) (ou un autre nom, vous êtes libre là-dessus), qui contiendra une sous-commande [hdr](#) qui s'occupera d'abstraire la manipulation de l'en-tête de la cartouche.

```
% cartridge --help
Usage: cartridge [OPTIONS] COMMAND [ARGS]...

  Cartridge SDK CLI entry

Options:
  --help  Show this message and exit.

Commands:
  hdr  Cartridge header interface
```

Et pour finir, la sous-sous-commande [dump](#) qui affichera le contenu d'un en-tête d'une ROM.

```
% cartridge hdr dump --help
Usage: cartridge hdr dump [OPTIONS] [GBA_FILES]...

  dump / display cartridge header information

Options:
  --help  Show this message and exit.
```

Voici un exemple d'affichage d'information de l'en-tête de ma ROM de référence:

```
% cartridge_hdr_dump super_mario_world_super_2.gba
super_mario_world_super_2.gba:
|-- entry
|   |-- valid: valid
|   |-- raw: 0xea00002e
|   |-- opcode: b 0xc0
|-- nintendo logo:
|   |-- status: valid
|   |-- debugging: disabled
|-- game title: SUPER MARIOB
|-- game code:
|   |-- code: AA2E
|   |-- date: 2001..2003 (old)
|   |-- language: USA/English
|-- marker code:
|   |-- id: 01
|   |-- developer: Nintendo
|-- fixed value: valid (96h)
|-- unit code: 00
|-- device type: 00h
|-- reserved: valid
|-- software_ver: 00h
|-- checksum:
|   |-- valid: valid
|   |-- rom: 8e
|   |-- our: 8e
```



Notez que vous pouvez ajouter autant de commandes que vous voulez. Il n'y a aucune contrainte à ce niveau-là.

Vous êtes libre pour le choix du langage pour la conception du SDK. Cependant, je vous recommande très chaudement le [Python \(3.10\)](#) couplé au framework [Click](#) pour la conception de la CLI. Dans tous les cas, utilisez un langage de haut niveau pour la conception d'outil, cela vous fera gagner un temps fou et vous facilitera grandement la vie lorsque nous attaquerons la mise en place du système de compilation ou de conversion d'images.

Bien sûr, je ne vous jette pas sous le bus. Le header a été documenté depuis ces 25 dernières années et vous pouvez en trouver [une documentation sur GBATEK](#) qui est très bien (quoiqu'un peu flou sur les bords, ce qui m'arrange pour mon sujet). Je vous recommande fortement de tester sur des ROM "officielles", cela vous permettra peut-être de récupérer des informations sensibles dont vous aurez besoin pour contourner la sécurité de la console...



Pas la peine de supporter les ROM "multiboot" qui contiennent un header plus compliqué. Au vu de nos besoins, vous n'en aurez pas besoin.

Bootstrap

Maintenant que nous avons de quoi récupérer et checker le header du jeu, il est temps de concevoir le deuxième outil qui nous permettra de construire une ROM valide.

Avant ça néanmoins, il nous faut un début de programme qui fait quelque chose, c'est pourquoi je vais vous fournir un bout de code assembleur qui affiche un écran rouge.

```
.text
.global _bootstrap

// Bootstrap code, aggressively request a RED screen
// @notes
// - do not try this code on real hardware unless you want to destroy your
//   batteries' life...
// - this code is located right after the CARTRIDGE header
// - no interruption handler setup...So, aggressively mask all possible
//   interrupt to avoid hardware crash
// - no stack setup, be careful
_bootstrap:
    // screen configuration
    // @note
    // - select the video mode 3 which is configure a basic VRAM
    // - enable the screen display background 2 (as requested with mode 3)
    // @out
    // - r3 -> new display register configuration
    // - r4 -> old display register configuration
    ldr    r0, =reg_dispcnt_addr
    ldr    r0, [r0]
    ldr    r4, [r0]
    ldr    r2, =reg_dispcnt_config
    ldr    r2, [r2]
    strh   r2, [r0]
    ldr    r3, [r0]

    // VRAM color fill
    // @note
    // - very, very, very slow VRAM update
    // - color the whole VRAM in RED
    // - screen size (w=240,h=160)
    ldr    r0, =vram_addr
    ldr    r0, [r0]
    mov    r2, #31
    mov    r1, #(240*160)
color_loop:
    strh   r2, [r0]
    sub    r1, #1
    add    r0, #2
    cmp    r1, #0
    bne    color_loop

    // Waiting loop of death
death_loop:
    b      death_loop
    nop

//---
```

```
// Data area
// ---

.balign 4
vram_addr:
    .long 0x06000000
reg_dispcnt_addr:
    .long 0x04000000
reg_dispcnt_config:
    .word 0x0403
```

Vous n'avez rien à faire dans cette section, si ce n'est de copier-coller le code au-dessus dans un fichier `bootstrap.s` (oui, un grand `s` et non un petit `s`, nous verrons ça plus tard) que nous compilerons lors de la prochaine étape. Pour l'instant, je vais faire un rapide détour pour vous expliquer un peu plus en détail le fonctionnement de la mémoire de la machine.

todo: schema MMU

Je vais partir du principe que vous êtes assez à l'aise avec les pointeurs et j'espère ne rien vous apprendre si je vous explique qu'un pointeur permet d'accéder à un endroit très précis dans la mémoire. Cependant. Lorsque vous utilisez un pointeur sur votre machine, vous manipulez ce qu'on appelle de la mémoire "virtuelle" (première partie du schéma). C'est-à-dire que quand vous tentez d'écrire ou de lire via votre pointeur (virtuel), la machine va aller demander au MMU (*Memory Management Unit*), un composant physique qui s'occupe de faire la translation entre mémoire virtuelle et mémoire physique, d'aller lire dans "la vraie mémoire" (physique).

todo: schemas processus

L'intérêt d'utiliser un MMU de nos jours est en grande partie lié à la sécurité, car le MMU peut mettre des droits sur certains endroits. Cependant, il est aussi surtout utilisé pour faire croire à un processus qu'il est contenu entièrement et de manière continue en mémoire, alors que ce n'est pratiquement jamais le cas. Cela permet aussi d'offrir la possibilité d'exécuter des binaires qui font des tailles gigantesques, dépassant toutes vos mémoires de travail (RAM généralement) présentes sur votre machine en lui donnant l'illusion d'être constamment chargée.

Une stratégie phare avec l'utilisation du MMU, c'est de faire croire à la personne qui demande de la mémoire qu'il a eu ce qu'il voulait en spéculant qu'il ne l'utilise jamais. En revanche, c'est uniquement dès que le demandeur tente de lire, écrire ou exécuter la mémoire que là, le MMU (le kernel pour être plus précis), daigne vraiment allouer de la mémoire.



Typiquement, si vous avez un processeur 32 bits, les adresses (pointeurs) peuvent aller de `0x00000000` à `0xffffffff`, ce qui représente 4 Go. Faites un programme qui boucle simplement sur un `malloc()` qui demande de plus en plus de taille. Vous allez observer qu'il échouera aux alentours de ~4Go. Maintenant faites le même test, mais à chaque fois, tentez simplement de lire ou écrire dans la mémoire. Vous allez observer que le `malloc()` échoue beaucoup plus tôt puisqu'on lui demande "vraiment" de la mémoire.

todo: schemas memory map

Revenons à notre sujet principal, la GBA. Il n'y a pas de notion de mémoire virtuelle sur cette machine (pour tout un tas de raisons), ce qui signifie que quand vous utilisez un pointeur, vous allez taper directement sur le matériel, il n'y a pas de garde-fou. Donc, imaginez que vous avez un bug dans votre programme qui manipule un porteur qui, par hasard, lorsque vous écrivez dedans, écrase la mémoire de la machine (le BIOS, admettons)... et bien vous cassez définitivement la console.



C'est ce qui est arrivé à [AHelper](#), un développeur avant-gardiste des années 2012 sur calculatrice (proche de la GBA), lorsqu'il essayait d'implémenter un bout de code qui chargeait en mémoire des librairies dynamiques en touchant au MMU. L'histoire reste floue, mais il a raté son coup en manipulant des porteurs et la machine a corrompu et/ou effacé le "boot" de la machine, l'empêchant ainsi de démarrer. La machine ne peut pas être réparée sans intervention matérielle très poussée et a donc été considérée comme "morte" (ou brickée)

Il faut donc être parfaitement conscient du comportement de la machine à chaque ligne de code que vous écrivez. C'est généralement après ce genre de projet que vous comprenez exactement l'intérêt du langage C et le "pourquoi" il a été conçu : contrôler des machines (et rien d'autre).

Avec le temps et l'expérience, vous allez vous rendre compte qu'en écrivant du C, modulo certaine sorcellerie, vous pouvez savoir avec une grande précision les instructions assembleur générés...et donc le comportement du matériel à un instant donné.

Je vous rassure tout de même, il est honnêtement peu probable que vous arriviez à casser une machine, il y a énormément de protection matérielle qui empêche de "bricker" votre machine ou de la réparer après coup.

Repassons un peu sur l'aperçu de la "map mémoire" de la GameBoy Advance présentée lors du précédent schéma. Vous pouvez voir qu'elle est découpée en "zones" chacune avec des caractéristiques/ objectifs différents. Par exemple, si on reprend rapidement le code assembleur que je vous ai fourni au début de ce chapitre, vous pouvez voir que la "variable" `vram_addr` utilise l'adresse `0x06000000` qui, si on en suit le tableau, pointe directement dans la VRAM ([Video RAM](#), la mémoire vidéo qui est affichée à constamment l'écran), ce qui semble cohérent pour faire un écran tout rouge.

todo: ecran anti-piratage Game Pak

Une section qui va nous intéresser actuellement dans cette cartographie de la mémoire, c'est la section [Game Pak](#) qui est le terme technique utilisé par Nintendo pour "cartouche de jeu". C'est donc ici que le code de la cartouche sera accessible. Ce qui nous donne, avec toutes les informations glanées depuis le début de ce sujet, le flow de démarrage suivant :

1. allumage de la console, le BIOS démarre automatiquement
2. le BIOS s'occupe d'initialiser la machine (initialiser tous les composants matériels)
3. le BIOS vérifie ensuite que la cartouche respecte la sécurité imposée (le header)
4. le BIOS affiche l'animation de démarrage

5. le BIOS saute à 0x08000000 et donne la main à la cartouche

Nous parlerons plus en détail de pourquoi savoir le fonctionnement exact du flow de démarrage nous est absolument capitale pour commencer à développer plus tard. Néanmoins, vous devriez maintenant comprendre pourquoi le premier champ du header est une simple instruction saut. Cette partie est critique puisque la console l'exécutera quoi qu'il arrive.

Parlons maintenant de la dernière notion à aborder, et qui est aussi le titre de cette section, le bootstrap.

Le bootstrap est littéralement le nom du premier bout de code "développeur" qui est exécuté juste après le BIOS et qui pourrait s'apparenter à l'étape numéro 6 dans le flow de démarrage que l'on vient de poser.

Son rôle est de mettre une pile (stack) "utilisateur" (car il n'y en a pas forcément une de mise), initialiser certaines mémoires, initialiser les périphériques matériels pour lever des ambiguïtés de configuration (nous en parlerons lors de la partie deux du sujet) et mettre en place suffisamment de choses pour que du code écrit dans un autre langage puisse être exécuté, ... Pour faire simple, c'est notre BIOS / code de démarrage à nous.

Le bootstrap que je vous ai fourni (le bout de code assembleur) ne fait rien d'autre que mettre l'écran en rouge. Je ne me suis même pas embêté à mettre en place une pile (stack) ou à correctement initialiser la machine. Nous verrons plus tard dans le sujet comment écrire un bootstrap plus élaboré.

Compilation

Bon, j'en dis des choses... c'est bien beau, mais il serait temps de passer à quelque chose de concret : la compilation.

Pour générer un binaire sur une la GameBoy Advance, à votre niveau, il faut simplement savoir que le processeur utilisé est le [ARM7TDMI](#). C'est un processeur ARM avec quelques fonctionnalités particulières qui ne nous intéressent pas particulièrement actuellement. Nonobstant, cela nous permet de savoir quel compilateur on peut utiliser, parce que si vous utilisez [gcc](#) ou [clang](#) sur votre machine, ils généreront des binaires...pour votre machine et c'est tout.

Il nous faut donc un compilateur "spécial" capable de supporter le processeur de la GBA et nous avons de la chance, puisqu'il en existe un que je vous invite à installer (vous devez installer les deux paquets mentionnés):

- ✓ [arm-none-eabi-gcc](#)
- ✓ [arm-none-eabi-newlib](#) (pour avoir accès à certains en-têtes particuliers))



Pour votre culture générale, que signifient les quatre parties au nom du compilateur ? - [arm](#) : indique l'architecture visée, ici ARM - [none](#) : indique le système d'exploitation, ici aucun - [eabi](#) : indique que nous n'utilisons pas d'interface pour les appels système ([Application Binary Interface](#)), on tourne directement sur le matériel

Maintenant que nous avons un compilateur, je vous donne tout le flow de compilation de notre mini-application :

```
arm-none-eabi-gcc -mcpu=arm7tdmi -mlittle-endian -nostdlib -T gba.ld -o test.elf bootstrap
.S
arm-none-eabi-objcopy -O binary test.elf test.bin
cartridge hdr gen -o test.gba -b test.bin
```

Si vous tentez d'effectuer les commandes, ça ne fonctionnera pas ☹ La honte cosmique si vous avez simplement copier-coller les lignes!

On va faire une passe pour décortiquer la première commande :

- ✓ [-mcpu=arm7tdmi](#) : indique que l'on veut générer du code pour le processeur [ARM7TDMI](#)
- ✓ [-mlittle-endian](#) : indique que l'on veut générer du code en little-endian (pas obligatoire, car mis automatiquement + ignoré par GCC)
- ✓ [-nostdlib](#) : indique que l'on ne veut pas de librairie standard (même si on a newlib)
- ✓ [-T gba.ld](#) : indique que l'on veut utiliser le linker script [gba.ld](#)... que vous n'avez pas encore

Ensuite, concernant les deux autres commandes, le [objcopy](#) prend le fichier au format [ELF](#) qui est utile pour

les systèmes d'exploitation, mais parfaitement inutile pour nous qui n'en avons pas, et on y extrait uniquement le code binaire en brut.

Et pour finir, on prend le binaire et on y ajoute notre header pour en faire un vrai ROM. Vous aurez compris que cette commande sera à implémenter à la fin de cette première partie.

todo: schemas flow de compilation

En parlant de linker script, je vais tenter de vous expliquer très rapidement son intérêt et pourquoi on en utilise un.

Le rôle d'un linker script, c'est d'indiquer au "linker" (la toute dernière étape lors de la génération d'un programme), comment "structurer" notre binaire. Quelles fonctions vont se trouver à quelle adresse en mémoire dans le but de respecter la cartographie des mémoires de la machine (la memory map). Je suis conscient que mon explication reste très floue, mais nous aurons l'occasion d'en parler beaucoup plus en profondeur lors de la partie 3 du sujet.



À savoir que vous utilisez forcément un linker script sans le savoir, car il est caché et ajouté automatiquement lorsque vous appelez `gcc` sur votre machine (pareil pour `-lc` de la librairie C qui est ajouté dans votre dos à chaque fois (d'où le `-nostdlib`)). Vous pouvez faire apparaître le linker-script de votre machine avec `gcc -Wl,--verbose`.

```
/* custom linker script in order to generate a GameBoyAdvance binary */
OUTPUT_FORMAT("elf32-littlearm", "elf32-bigarm", "elf32-littlearm")
OUTPUT_ARCH(arm)
ENTRY(_bootstrap)

/* address only the cartridge memory area
 * @notes:
 * - skip the cartridge header
 * - other memory area will be cover soon.. */
MEMORY {
    rom (rx) : ORIGIN = 0x080000c0, LENGTH = 32M
}

/* aggressively keep code binary and force it to be located at the start of the ROM */
SECTIONS {
    .text : {
        KEEP(*(.text));
    } > rom
}
```

Voici le fameux `gba.ld`. Comme vous pouvez le voir, il est extrêmement minimaliste comparé à celui de votre machine et dit basiquement au compilateur : ne fait rien d'autre que de mettre tout le code assembleur à l'adresse `0x080000c0...` qui se trouve être, si on regarde notre tableau encore une fois, juste après la section du `Game Pak`. Vous devriez commencer à comprendre pourquoi on ne commence pas à `0x08000000` directement, mais avec un décalage de `0xc0` octets.

Avec cette dernière pièce du puzzle, vous devez avoir suffisamment d'informations pour au moins réussir à générer le binaire brut du projet. D'ailleurs, vous pouvez vérifier par vous-même en utilisant `arm-none-eabi-objdump -m arm7tdmi -b binary -D test.bin` pour vérifier que les instructions sont bien celles qui sont écrites dans le `bootstrap.S`. Nous aurons l'occasion de reparler de la notion de "debug" plus tard.

Par contre...vous avez la même impression que moi ? Ce n'est pas complètement chiant de tout écrire à la main ? Pourquoi ne pas ajouter une entrée dans notre superbe SDK ? Ajouter la sous-commande `cartridge build` qui buildera automatiquement votre projet. (Je conçois que là, ce n'est pas forcément intéressant d'avoir une commande qui va juste lancer votre Makefile à votre place...mais, pour la suite, vous verrez un intérêt (et encore plus avec le temps)).



Faites un simple enrobage autour de Make pour l'instant et ignorez la génération du header que nous n'avons pas encore implémentée. D'ailleurs, en parlant de ça...

Cartridge

Maintenant que nous avons un binaire pur, il ne manque plus qu'à générer la ROM finale. En soi, nous savons déjà comment fonctionne le header...reste plus qu'à faire le processus de génération.

Implementer la commande suivante:

```
% cartridge_hdr_gen --help
Usage: cartridge_hdr_gen [OPTIONS]

    header generation

Options:
  -b, --binary FILE  input binary file [required]
  -o, --output FILE  output filename [required]
  -n, --name TEXT    internal cartridge name (default: filename)
  --help            Show this message and exit.
```



Vous pouvez ajouter d'autres flags pour le fun si vous voulez. Je vous recommande aussi d'utiliser le `cartridge_hdr_dump` pour vous assurer que votre génération est bonne.

Émulation

Maintenant reste la partie "fun" du projet : l'émulation.

Comme on vise à faire du bas niveau ainsi qu'un "observateur" de matériel, il nous faut un émulateur qui est orienté pour faire de l'observation matérielle. Un des émulateurs phares pour faire du développement du GBA s'appelle `no$gba`, malheureusement, il n'est pas disponible sur Linux et son interface reste extrêmement rudimentaire.

C'est pourquoi j'ai trouvé l'émulateur `ayyboy-advance` qui reste très pertinent dans son design pour faire ce que l'on souhaite faire. Cependant, l'outil est "mal conçu" sur certains points et fonctionne uniquement sur de très grands écrans (la taille de la fenêtre est totalement fixée à la main, ...). C'est pourquoi, j'ai pris sur moi de forker le dépôt et d'y ajouter des fonctionnalités et des correctifs spécifiques à notre sujet. Je vous invite donc fortement à utiliser ma variante du projet (et à y participer si le cœur vous en dit).

todo: screenshot

Parmi les micro-fonctionnalités que j'ai ajoutées (en plus de quelques fixes), vous avez un flag `--debugger` qui permet directement d'entrer en mode "débugueur", ce qui devrait grandement vous être utile par la suite.



Le fork est encore expérimental, je n'ai pas encore eu le temps de corriger certains bugs de GUI pour qu'il puisse être complètement utilisable dans son ensemble. Néanmoins, il reste pratique pour notre sujet et je continuerai à l'améliorer avec le temps. Je vous tiendrai au courant des avancées.

Votre objectif est donc de réussir à build le projet et à lancer votre jeu sur l'émulateur. Théoriquement, vous ne devriez pas croiser de problème particulier. Il vous restera plus qu'à ajouter une commande dans votre SDK pour lancer automatiquement l'émulateur à la fin de chaque compilation.



Vous pouvez simplement ajouter une règle dans votre Makefile (eg `make emu`), mais vous pouvez aussi créer une nouvelle sous-commande qui s'occupera de récupérer l'émulateur, le build et le lancer. C'est comme vous le sentez pour l'instant.

Voilà! Nous avons maintenant de bonnes bases pour entrer un peu plus en profondeur sur la programmation bas-niveau!

À suivre...

Part II - Démarrage

Maintenant que nous avons un début d'environnement de développement avec notre SDK et notre écran rouge, il est grand temps de commencer à regarder droit dans les yeux le code de démarrage que je vous ai fourni.

L'idée de cette deuxième partie est de laisser un peu la conception d'outil à part pour nous concentrer sur la mise en place d'un environnement d'exécution "saint". Actuellement, le code que je vous ai fourni fonctionne un peu "par chance", du moins avec énormément de supposition matérielle et ne peut fonctionner qu'en assembleur "pure".

Notre but ici sera simplement de nettoyer notre base de code et "porter" le code relatif à l'écran dans une forme plus appropriée : un driver écrit en C.

Cette seconde partie va donc couvrir :

1. l'écriture d'un bootstrap plus évolué
2. l'écriture d'un début de driver pour l'écran
3. le début d'une API graphique

Je préfère être franc. Cette partie risque encore d'être assez frustrante étant donné que nous allons simplement "porter" le code actuel de l'écran rouge vers une base beaucoup plus solide. Et, comme pour la partie précédente, vous allez voir qu'on va brasser énormément de notions super techniques et complexes pour...l'exact même résultat.

Je vous rassure, une fois que cette partie sera passée, on pourra (presque) commencer à faire tout ce qu'on veut avec le matériel. En attendant, c'est un passage obligé pour préparer la suite.

Bootstrap (pour de vrai cette fois)

Si on récapitule le flow du démarrage de la console qu'on a vu lors de la première partie :

1. la console démarre
2. le BIOS initialise le matériel
3. le BIOS vérifie que la cartouche est valide via son header
4. le BIOS saute à `0x08000000` et donne le relai à la cartouche
5. la cartouche contient un simple saut vers la fin de son en-tête (généralement `0x080000c0`)
6. début d'exécution du jeu

Maintenant que nous savons d'où nous venons, il est très important de faire un constat de ce que j'appelle "le contexte d'exécution". Typiquement, vous vous mettez à la première instruction du `bootstrap.S` et vous vous posez les questions suivantes:

- ✓ Est-ce que je peux me faire interrompre ?
- ✓ Si oui, à quel point je suis impacté ?
- ✓ Quels sont les registres disponibles ou non ?
- ✓ Quel est le contexte du processeur ?
- ✓ Quel est le statut de la stack ?

Les deux premiers points concernant les interruptions sont les plus critiques à prendre en compte. On ne parlera pas tout de suite du fonctionnement profond des interruptions matérielles, mais il faut quand même être conscient que votre code peut être interrompu (matériellement) à n'importe quel moment (instruction).

Prenons cet exemple de fonction mal sécurisée qui inverse la couleur d'un pixel à une position précise:

```
static void dpixel_invert(u8 x, u8 y)
{
    lcd_configure_cursor(x, y);
    u16 color = lcd_get_pixel();
    lcd_set_pixel(color ^ 0xffff);
}
```

Admettons que vous vous faites interrompre entre le `lcd_get_pixel()` et le `lcd_set_pixel()` et que l'interruption appelle elle aussi cette fonction, nous pouvons nous retrouver avec ce flow d'exécution:

1. `color = lcd_get_pixel(); //color = 0xaabb`
2. (interruption) `color = lcd_get_pixel(); //color = 0xaabb`
3. (interruption) `lcd_set_pixel(color ^ 0xffff); //color = (0xaabb ^ 0xffff)-> color = 0x5544`
4. `lcd_set_pixel(color ^ 0xffff); //color = (0xaabb ^ 0xffff)-> color = 0x5544`

Alors que le flow attendu est le suivant :

1. `color = lcd_get_pixel(); //color = 0xaabb`
2. `lcd_set_pixel(color ^ 0xffff); //color = (0xaabb ^ 0xffff)-> color = 0x5544`
3. (interruption) `color = lcd_get_pixel(); //color = 0x5544`
4. (interruption) `lcd_set_pixel(color ^ 0xffff); //color = (0x5544 ^ 0xffff)-> color = 0xaabb`

Nous nous retrouvons donc avec la "mauvaise" couleur en finalité, qui pourrait provoquer des glitches visuels.

Ce genre de problématique peut s'avérer terrible si vous n'avez pas de protection pendant la manipulation de registres périphériques. Votre code pourrait planter de manière non-déterministe. C'est-à-dire qu'il pourra planter aléatoirement sans que vous arriviez à reproduire le bug. Avoir un comportement non-déterministe est la dernière chose que vous voulez avoir lorsque vous développez.



Anecdote personnelle concernant un bug d'interruption lors de l'écriture d'un de mes premiers kernel. Je faisais ultra gaffe à ce genre de comportement, mais je me suis retrouvé avec des plantages occasionnels dans noyau. J'ai mis presque un mois à trouver l'origine du problème et à le corriger. Le bug était simple, il y avait 2 instructions qui n'étaient pas correctement protégées...sur ~150000 instructions.

Pour vous protéger contre ce genre de comportement, vous devez mettre en place ce qu'on appelle un contexte "atomique". Les contextes "atomiques" vont bloquer toute possibilité d'interruption **matérielle** le temps d'effectuer une action.

```
static void display_pixel_invert(u8 x, u8, y)
{
    cpu_atomic_start(); // bloque les interruptions
    lcd_configure_cursor(x, y);
    u16 color = lcd_get_pixel();
    lcd_set_pixel(color ^ 0xffff);
    cpu_atomic_end(); // remets les interruptions
}
```

Les opérations "atomiques" de cette nature sont beaucoup plus puissantes et dangereuse que l'utilisation de `mutex()` (ou autre), car on va basiquement demander au processeur d'ignorer **physiquement** les interruptions matérielles.

todo: image schema interruption materiel



Il faut bien prendre en compte qu'une interruption matérielle est prioritaire et ne rend la main uniquement quand elle a fini!

Sur la GBA, le registre `CPSR|IME|IE` (bits 0-4), permet de "masquer" les interruptions matérielles. L'idée est donc que `cpu_atomic_start()` masque les interruptions et `cpu_atomic_end()` réactive les interruptions en

manipulant ce registre.

A notre stade du projet, on va simplement masquer les interruptions. Et ça sera à vous de le faire en modifiant le `bootstrap.S`.



Avoir le principe de "réentrance" en tête est donc super important. Déjà pour tous les projets où vous allez manipuler des threads, mais aussi parce que dans notre cas, si on foire notre coup, c'est la machine qui plante physiquement (pouvant aller jusqu'à la casser).

Le second point à faire très gaffe, c'est le contexte du processeur.

Il faut savoir que tous les processeurs ont des registres de statut qui leur permettent de les contrôler ou d'avoir des informations sur leurs états. Par exemple, on peut savoir si on tourne en mode "kernel", "superviseur" ou en "utilisateur".

Cette notion est importante, car si vous vous trouvez en mode "user", certaines instructions ou zones de la mémoire peuvent être refusées par le processeur (cela va générer une exception, qui est une catégorie bien précise des interruptions matérielles qu'on abordera dans une autre partie).

Et le dernier point qui est important dans notre cas, c'est l'état de la stack. Est-ce qu'on a une stack ? Est-ce qu'on a de la place sur la stack ? Où est-ce qu'elle se trouve ? Officieusement, la stack est mise en place par le BIOS vers la fin de la `WRAM` à l'adresse `0x03007F00`.



Pour la postérité pédagogique, on va simplement dire qu'on n'a pas de stack et qu'on va devoir en mettre une manuellement.

Sur tous les points énoncés, aucune ne peut être répondue avec certitude dans notre contexte :

- ✓ On ne sait pas la configuration du processeur
- ✓ On ne sait pas si on a une stack
- ✓ On ne sait pas si on a des interruptions qui peuvent nous interrompre à tout moment.

Bien sûr, le BIOS est censé (théoriquement) nous mettre en place un minimum de choses pour qu'on puisse exister. Puis avec l'émulateur on peut voir quels registres ont quelle valeur au démarrage de la cartouche.

Néanmoins, on ne sait pas à quel point l'émulateur a raison et concernant le BIOS, on n'a aucune certitude de rien, puisqu'on n'a aucune idée de ce qu'il fait réellement (c'est une boîte noire). La première chose à faire est donc de lever ces ambiguïtés matérielles.

Reprennez le code du [bootstrap.S](#) et ajouter directement:

- ✓ la désactivation des interruptions matérielles
- ✓ forcer la compatibilité des instructions en ARM et non THUMB
- ✓ mettre en place un stack utilisateur à l'adresse `0x03007F00`
- ✓ Écrire une documentation sur la configuration du processeur (commentaires)



Je vous conseille fortement de regarder le registre `CPSR` et ce qu'on appelle communément la "[calling convention](#)" pour savoir le rôle de tous les registres.



Concernant le deuxième point. Il faut savoir que le processeur supporte 2 types d'instructions : `THUMB` (16 bits) et `ARM` 32 bits. On verra un autre moment comment exploiter les deux modes.

Et le C dans tout ça ?

Maintenant que nous sommes marginalement certains de l'environnement d'exécution, il serait peut-être temps d'écrire le premier code en C, non ? Commençons par la fameuse `extern void main(void);` (on s'en fiche de retourner un nombre et nous n'avons pas besoin d'argument).

Pour nous aider à savoir ce qu'il nous faut pour pouvoir lancer notre `main()`, on va simplement observer comment fait GCC pour traduire une fonction en assembleur afin de faire la même chose de notre côté:

```
#include <stdbool.h>

int _gigaPaffDu67(int *count) {
    __asm__ volatile ("nop");
    return *count - 1;
}

void main(void)
{
    int count = 667;

    while(true) {
        count = _gigaPaffDu67(&count);
        if (count <= 0)
            break;
    }
}
```

Pourquoi un code aussi compliqué ? Pour deux raisons.

La première c'est qu'on aimerait savoir ce que la fonction `main()` fait réellement une fois traduite en assembleur, mais on sait aussi qu'on va devoir appeler la fonction `main()` depuis l'assembleur...et on ne sait pas comment faire. Donc on fait aussi un appel de fonction basique pour voir comment le compilateur implémente l'appel d'une fonction.



Le code est volontairement "complexe" pour "forcer" le compilateur à se perdre dans ces optimisations. Si je n'avais pas utilisé de pointeur ou d'instruction assembleur, GCC aurait tout simplement fusionné les deux fonctions (`inline`) et on aurait perdu l'appel de fonction ainsi que beaucoup d'information.



La notion de `__asm__ volatile ("nop");` dans la fonction du `_gigaPaffDu67` et une notation assez "freestyle" pour injecter du code en assembleur en plein milieu de la fonction. Le `volatile` est là pour dire au compilateur : "touche pas à cette instruction, je maîtrise".

Pour compiler c'est pas complexe, on n'a même pas à s'embêter avec le linker script, on va simplement taper:

```
arm-none-eabi-gcc -mcpu=arm7tdmi -c tmp.c
arm-none-eabi-objdump -j .text -D tmp.o
```

La génération d'un fichier objet ne demande pas de linker script puisqu'on ne crée pas un ELF/binaire **final**. Et comme les fichiers `.o` sont basiquement des fichiers ELF qui contiennent toutes les infos, on n'a même pas à s'embêter avec des flags pour décompiler les instructions, `objdump` se débrouille!



Je vous laisse voir exactement à quoi sert le `-j .text`, mais on l'abordera au prochain chapitre.

Nous nous retrouvons donc avec des lignes d'assembleur qui ressemble à ça (et je vous invite très très fortement à avoir la documentation [des instructions](#) et [des registres](#) à côté de vous pour pouvoir suivre):

```
00000000 <_gigaPaffDu67>:
 0: e52db004 push {fp} @ (str fp, [sp, #-4]!)
 4: e28db000 add fp, sp, #0
 8: e24dd00c sub sp, sp, #12
 c: e50b0008 str r0, [fp, #-8]
10: e1a00000 nop @ (mov r0, r0) ! notre `__asm__ volatile ("nop")` !!
14: e51b3008 ldr r3, [fp, #-8]
18: e5933000 ldr r3, [r3]
1c: e2433001 sub r3, r3, #1
20: e1a00003 mov r0, r3
24: e28bd000 add sp, fp, #0
28: e49db004 pop {fp} @ (ldr fp, [sp], #4)
2c: e12fff1e bx lr

00000030 <main>:
30: e92d4800 push {fp, lr}
34: e28db004 add fp, sp, #4
38: e24dd008 sub sp, sp, #8
3c: e59f3038 ldr r3, [pc, #56] @ 7c <main+0x4c>
40: e50b3008 str r3, [fp, #-8]
44: e24b3008 sub r3, fp, #8
48: e1a00003 mov r0, r3
4c: ebfffffe bl 0 <_gigaPaffDu67>
50: e1a03000 mov r3, r0
54: e50b3008 str r3, [fp, #-8]
58: e51b3008 ldr r3, [fp, #-8]
5c: e3530000 cmp r3, #0
60: da000000 ble 68 <main+0x38>
64: eaffffff b 44 <main+0x14>
68: e1a00000 nop @ (mov r0, r0)
6c: e1a00000 nop @ (mov r0, r0)
70: e24bd004 sub sp, fp, #4
74: e8bd4800 pop {fp, lr}
78: e12fff1e bx lr
7c: 0000029b muleq r0, fp, r2
```


Nous pouvons voir la première chose importante au début de chaque fonction : l'instruction `push` est présente. Ce qui nous indique, subtilement, que la stack est utilisée! Il nous fallait donc au moins une stack pour faire un appel `main()`! C'est étrange ce qu'on a mis en place le chapitre précédent. Purée, mais on pourrait croire que tout était prévu depuis le début et que je maîtrise mon sujet, qu'est-ce que je suis bon!

Vous pouvez aussi remarquer un truc assez tordu de la part de GCC : il n'affiche pas exactement les bonnes instructions. Typiquement, l'instruction `nop` est en réalité un `mov r0, r0`, car il n'existe pas d'instruction `nop` native sur le processeur. Pareil pour l'utilisation des registres `sp` ou `pc` qui sont des alias aux registres `r13` et `r15` respectivement.



Vous allez aussi voir passer pas mal de fois la notion du registre `fp...` qui n'existe tout simplement pas. Ce "registre" est en fait ce qu'on appelle un `frame pointeur`. C'est un mécanisme mis en place par GCC pour gérer les variables locales. On s'en fiche pour l'instant.

Maintenant l'appel de fonction.

Vous pouvez voir l'utilisation de l'instruction `bl <_gigaPaffDu67>`. Vous aurez aussi remarqué la présence du registre `lr` qui est aussi un alias utilisé par GCC pour désigner `r14` qui fait office de "Link Register" (voyez ça comme un registre qui contient une adresse pour du code).

Sauter dans une fonction, on a compris, mais qu'en est-il du retour de fonction (notre bon vieux "`return`") ? Vous pouvez voir (et lire dans la documentation), que pour sortir d'une fonction, il faut donc... utiliser l'instruction `bx lr` (soit `bx r14`) qui est un simple saut...Ce qui n'est pas forcément étrange puisqu'un `return` n'est rien d'autre qu'un saut en arrière.

Passons à l'exercice.

Vous allez devoir créer un fichier `main.c` qui contient une fonction `extern void main(void)` qui ne fait rien d'autre qu'une boucle infinie. Ensuite, vous allez devoir modifier encore une fois le fichier `bootstrap.S` pour appeler proprement la fonction `main()`.



Comment allez-vous réagir si jamais on sort du `main` ? Comment la console réagit lorsque l'on rend la main au BIOS ?

Si tout se passe bien, votre programme devrait planter! Si ce n'est pas le cas, c'est de la chance et nous allons voir pourquoi en détail au prochain chapitre.

Maintenant que nous avons un `main()`, pourquoi notre cartouche plante (ou presque)? Eh bien c'est à cause du linker script.

Comme je l'avais mentionné rapidement lors de la première partie, le linker script est un fichier qui donne des indications au linker. Le but étant de lui décrire comment construire le binaire final.

Il y a néanmoins deux types de "binaires" à dissocier : le format ELF et le binaire pur. C'est assez subtil, mais nous passons par les deux formes pour créer notre jeu (contrairement au programme "classique" qui passe uniquement par ELF).

La première forme concerne les ELF. Sans refaire toute l'histoire du format, il faut simplement savoir que c'est un type de fichier créé pour simplifier la vie des "loaders" qui sont des programmes qui ont pour objectif...de charger des programmes en mémoire et de les exécuter.

todo: schema ELF

Si on prend ce schéma, vous pouvez voir que le fichier est constitué d'un en-tête puis divisé en plein de "sections" (l'en-tête aidant à savoir quelle section se trouve où). Chaque section contient basiquement une adresse de chargement, des caractéristiques propres (taille, alignement en mémoire, ...) ainsi que des droits.

Parmi les sections les plus connues (et qui nous intéressent), nous pouvons citer :

- ✓ `.text` (exec) contient toute les instructions (le binaire **executable**)
- ✓ `.data` (read/write) contient toute les variable globale **initialisé**
- ✓ `.rodata` (read) contient toute les variables globale **constante**
- ✓ `.bss` (read/write) contient toute les variables globale **non-initialisé**

Quel est l'intérêt d'avoir découpé les informations en section plutôt que de tout avoir en un bloc ? Principalement pour la sécurité, car chaque section contient ses propres droit, mais aussi pour des raisons d'optimisation.

Typiquement, on remarque que la section `.bss` et `.data` contiennent basiquement les mêmes informations ainsi que les mêmes droits (lecture et écriture). Cependant, `.bss` contenant uniquement des données non initialisées, on ne stocke donc rien dans cette section! Ce qui nous donne des binaires beaucoup plus légers!



Certes on gagne de la place dans le binaire final, mais la place est bien réservée dans la mémoire. C'est simplement une optimisation de place.



Je vous recommande chaudement le livre "Linkers and Loaders" si jamais vous voulez approfondir ce trou de lapin abyssal que sont le fonctionnement interne des linkers.

Revenons à notre cas en analysant petit bout par petit bout notre linker script.

```
OUTPUT_FORMAT("elf32-littlearm", "elf32-bigarm", "elf32-littlearm")
OUTPUT_ARCH(arm)
ENTRY(_bootstrap)
```

Tout ce que vous voyez au-dessus, c'est pour le fichier ELF et les outils de GCC. Je vous invite à aller chercher ce que font chaque mot clé, mais le plus important reste `ENTRY(_bootstrap)` qui indique au "loaders" où est l'entrée du programme (ici la fonction `_bootstrap`, qui est définie dans le fichier `_bootstrap.S`).

```
MEMORY {
    rom (rx) : ORIGIN = 0x080000c0, LENGTH = 32M
}
```

Cette partie décrit les différentes "mémoires" disponibles pour les sections. C'est basiquement ici que vous allez ajouter des mémoires comme par exemple la `WRAM` ou la `VRAM`. C'est relativement simple : un nom, les droits, une adresse et une taille.

```
SECTIONS {
    .text : {
        KEEP(*(.text));
    } > rom
}
```

Et c'est ici que commence toute la malicerie malicieuse.

Cette partie décrit avec précision chaque "section" du programme. Si nous prenons le temps de regarder le code droit dans les yeux, on peut comprendre ceci:

1. nous descrivons la section `.text`
2. le `> rom` nous indique qu'on met la section `.text` dans la mémoire `rom` (donc `0x080000c0`)
3. tous les bouts de code qui demandent la section `.text` sont mis dans la section `.text`

Je sais très bien que la dernière phrase n'a aucun sens, mais c'est normal. En fait, le compilateur "attribue par défaut" des sections aux symboles qu'il croise (par exemple, s'il croise une fonction, il lui attribue la section `.text`, idem pour une variable globale non initialisée avec `.bss`). Nous, notre job est de prendre les attributions par défaut et de les mettre au bon endroit.



La notion de symbole est super importante, car employée de partout en bas-niveau. Voyez les symboles comme des fonctions, des variables globales, ... bref, tout ce qui peut être utilisé avec le mot-clé `extern` (publique).

Maintenant notre problème de crash.

Nous mettons absolument **tout** le code dans la section `.text`. Si on compile d'abord `main` et ensuite `_bootstrap`, la section `.text` ajoutera les fonctions dans l'ordre.

Autant pour le format ELF et les "loaders", on s'en tape puisqu'on précise explicitement que la première fonction à appeler c'est `_bootstrap...` mais dans notre cas, on n'a pas du tout de "loader" et on a un binaire pure à la fin. La console saute simplement à `0x080000c0` et s'en tape de savoir qu'il faut appeler une fonction en particulier!

Et du coup, comme `main()` est mis en premier... et bien c'est lui qui se trouve à `0x080000c0`! Vous pouvez afficher la cartographie des symboles mis par le linker avec cette commande (c'est la seule façon, à ma connaissance, pour debug le linker script):

```
arm-none-eabi-gcc -mcpu=arm7tdmi -Wl,-Map=gba.map -Wl,--print-memory-usage -nostdlib -T
gba.ld -o test.elf bootstrap.S main.c
```

Votre exercice est donc simple :

- ✓ Renomez la mémoire `rom` en `game_pack` (parce que c'est le "vrai" nom interne de la zone)
- ✓ Trouvez un moyen de forcer `_bootstrap` à être chargé en premier.
- ✓ Réussir à lancer le jeu sur l'émulateur



Vous n'avez pas à gérer les sections `.data`, `.rodata` ou `.bss` pour l'instant. On en reparlera en temps voulu.



Je vous invite fortement à vous balader avec la documentation `info ld` section 3 `Linker Script`. Pareil pour l'assembleur avec `info as` section 9.4 `ARM Dependent Features`.

Driver Ecran

Maintenant que nous avons passé la partie très technique et que nous pouvons écrire du code en C, il est grand temps de commencer à écrire notre premier driver : l'écran.

Un driver, c'est un mot un peu opaque pour parler d'un bout de code qui discute directement avec du matériel physique. Généralement, un driver fournit une "interface" pour faciliter l'utilisation du périphérique, et c'est exactement ce que l'on va mettre en place.

Dans notre cas, nous allons mettre en place les primitives / fonctions suivantes :

todo: tableau

- ✓ `extern void dinit(void)` : initialise l'écran
- ✓ `extern void dclear(u16 color);` : efface l'écran avec une couleur définie
- ✓ `extern void dpixel(i16 x, i16 y, u16 color);` : dessine un pixel à l'écran



Vous noterez l'utilisation de type "strict" comme `u8`, `u16`, `i8`, ... qui seront la norme par la suite.

Il faut maintenant qu'on parle de l'écran.

Sur la GBA, il existe plusieurs façons de dessiner à l'écran et la machine fournit plusieurs fonctionnalités différentes de dessin. Nous, nous allons nous concentrer sur la version "old-school" : dessiner tout à la main dans un premier temps, on verra ce que peut nous offrir la machine un autre moment.

Si on regarde le [bootstrap.S](#) que je vous ai fourni, on peut voir que je manipule le registre `DISPCNT` qui est, si on en croise la documentation, le registre de contrôle pour la LCD. A vous de comprendre ce que je fais pour le répliquer.

Votre but est donc simple : porter le code assembleur qui manipule l'écran en C en Implementant les fonctions listées plus haut. Vous devez écrire une "documentation" au-dessus de chaque fonction (via les commentaires) qui explique ce que vous tentez de faire, comment et pourquoi.

Petit tips.

Je vous recommande l'utilisation du mot-clé `volatile` lorsque vous manipulez des pointeurs qui font référence à des registres. Ce mot-clé permet de forcer le compilateur à constamment utiliser les pointeurs.

Par exemple ce code:

```
void _test(void)
{
    u8 volatile *test = (void*)0xc0ffee;
    u8 *test_volatile = (void*)0xdeadbeef;
```

```

// non volatile
// -> tout le code sera remplacé par `*test_0 = 0x55`
*test = 0x11;
a = *test;
*test = a | 0x44;

// volatile
// -> le code sera exécuté tel quel
*test_volatile = 0x11;
a = *test_volatile;
*test_volatile = a | 0x44;
}

```

C'est très contre-intuitif, mais le compilateur cherchera constamment à faire le moins d'effort possible pour aller plus vite.

Sans le qualificatif `volatile`, les lectures et écritures dans les pointeurs seront réduites à une écriture simple, alors qu'avec le `volatile`, toutes les opérations seront gardées.

Bon courage!

```

/* Packed structures. I require explicit alignment because if it's unspecified,
GCC cannot optimize access size, and reads to memory-mapped I/O with invalid
access sizes silently fail - honestly you don't want this to happen */
#define MYPACKED(x)    __attribute__((packed, aligned(x)))

/* shorthand strict type */
typedef unsigned int    uint;
typedef uint8_t         u8;
typedef uint16_t        u16;
typedef uint32_t        u32;
typedef uint64_t        u64;
typedef int8_t          i8;
typedef int16_t         i16;
typedef int32_t         i32;
typedef int64_t         i64;

/* Giving a type to padding bytes is misleading, let's hide it in a macro */
#define pad_nam2(c) _ ## c
#define pad_name(c) pad_nam2(c)
#define pad(bytes)  uint8_t pad_name(__COUNTER__)[bytes]

/* word_union() - union between an uint16_t 'word' element and a bit field */
#define word_union(name, fields) \
    union { \
        uint16_t word; \
        struct { fields } MYPACKED(2); \
    } MYPACKED(2) name

//---
// GBA LCD peripheral. Refer to:
// "GBATEK : LCD I/O Interrupts and Status"
//---
typedef struct {
    // I/O configuraton

```

```

word_union(DISPCNT,
    u16 BG_MODE      :2; // Video Mode
    u16               :1; // reserved
    u16 DFS          :1; // Display Frame Select
    u16 HBIF         :1; // H-Blank Interval Free
    u16 OCVF         :1; // OBJ Character VRAM Mapping
    u16 FB           :1; // Force Blank
    u16 BG0          :1; // Enable Screen Display Background 0
    u16 BG1          :1; // Enable Screen Display Background 1
    u16 BG2          :1; // Enable Screen Display Background 2
    u16 BG3          :1; // Enable Screen Display Background 3
    u16 OBJ          :1; // Enable Screen Display OBJ
    u16 WDF0         :1; // Window Display Flag 0
    u16 WDF1         :1; // Window Display Flag 1
    u16 WDFOBJ       :1; // Window Display Flag OBJ
);

// DISPGW : undocumented register, lets skip it
pad(2);
/* word_union(DISPGW,
    u16 SWAP         :1; // Enable green SWAP
    u16               :15; // reserved
); */

// General LCD Status
word_union(DISPCNT,
    u16 VBF          :1; // V-Blank Flag
    u16 HBF          :1; // H-Blank Flag
    ...
);
...
} MYPACKED(2) GBA_lcd_t;

/* In order to avoid manipulating raw pointer all time in the file, use a generic define
*/
#define GBA_LCD (*(volatile GBA_lcd_t *)0x04000000)

```

Partie III - Glyph

Nous avons enfin un environnement de développement ainsi qu'un environnement d'exécution qui commence à ressembler à quelque chose. Il est donc temps de mettre en place un "environnement de debuggage" afin qu'on puisse commencer à découvrir la machine.

La notion "d'environnement de debuggage" est vraiment grossière, dans notre cas, cela va simplement se résumer à réussir à afficher du texte à l'écran.

Reussir à faire un rendu graphique d'un texte est une notion extrêmement complexe sur les ordinateurs modernes, comme en témoigne ce [blog de Zed](#) ainsi que [cette vidéo](#). Heureusement, dans notre cas, le rendu de texte peut être grandement simplifié.

Cependant, nous allons avoir besoin d'un outil pour convertir notre "font" (qui sera une simple image créée sur Gimp), en format binaire optimisé pour nos besoins. C'est donc le retour de la conception d'outils pour notre SDK. Je vous rassure, il y aura quand même une petite partie technique à la fin.

Cette partie va donc concerner les thématiques suivantes :

- ✓ mise en place d'un algorithme de dessin
- ✓ conception d'outil de conversion d'image vers du code C
- ✓ du parsing
- ✓ début de visualisation technique

C'est basiquement la dernière partie casse-pied où on tourne autour de la machine avant de pouvoir pleinement la disséquer et comprendre ce qu'il se passe.

Comme rapidement expliqué lors de l'introduction à cette partie, dessiner du texte est une tâche extrêmement complexe pour les ordinateurs actuels. Cela est principalement dû aux millions d'écrans de taille et de types différents qui se trouvent dans la nature.

Actuellement, les "fonts" récentes contiennent, pour faire simple, une formule mathématique type courbe de Bezier qui explique "comment dessiner chaque lettre". L'intérêt derrière ce choix, plutôt qu'une simple image, c'est que l'on peut choisir la taille des lettres sans aucune perte de qualité (ce qui est très pratique au vu de la variété de taille d'écran qui existe).

Il faut donc, pour chaque lettre que vous voulez dessiner, être capable de calculer sa courbe de dessin ; appliquer des correctifs pour que ça rende bien sur votre écran ; corriger la couleur ;... tout ça en étant le plus rapide possible. C'est un processus extrêmement complexe et technique.



Bien sur, il existe beaucoup de technique pour rendre le dessin d'une lettre pratiquement instantané. Comme faire le rendu en une fois dans la mémoire et simplement "copier/coller" le rendu là où il y a besoins.

Dans notre cas, nous avons une petite taille d'écran fixe de 240*160 pixels. Nous n'avons donc pas à nous embêter à supporter de "font" avec une taille dynamique. Nous allons simplement dessiner manuellement des caractères de taille fixe, pixel par pixel.

todo: image font

La stratégie est donc la suivante : On va simplement représenter la table ASCII dans un gros rectangle qu'on va diviser en sous-rectangle (ici de 8 par 12 pixels) qui contiendront les dessins de chaque lettre. Il nous restera plus qu'à convertir chaque petit rectangle en donnée utilisable pour faire notre dessin.

Concernant la font que je vous fournis, les informations de géométrie sont représentées par des couleurs différentes:

- ✓ **rouge** : espace autour de chaque lettre
- ✓ **violet** : La ligne d'affichage. Utile pour les lettres qui "débordent en bas" (y, g, ...)
- ✓ **noir** : pixel à encoder
- ✓ **autre** : à ignorer

Comme un caractère est simplement un rectangle avec des pixels allumés ou éteints, on va compacter chaque zone en partant du principe que chaque pixel représente un bit unique. Ce qui nous donne, pour le caractère "5", la conversion suivante:

```
// ----- -> 0b00000000 -> 0x00
// #####- -> 0b11111100 -> 0xfc
// ##----- -> 0b11000000 -> 0xc0
// ##----- -> 0b11000000 -> 0xc0
// #####- -> 0b11111100 -> 0xfc
// -----##- -> 0b00000110 -> 0x06
// -----##- -> 0b00000110 -> 0x06
// -----##- -> 0b00000110 -> 0x06
// ##---##- -> 0b11000110 -> 0xc6
// -#####- -> 0b01111100 -> 0x7c
// ----- -> 0b00000000 -> 0x00
// ----- -> 0b00000000 -> 0x00
u8 const five_bmp[] = {0x00, 0xfc, 0xc0, 0xc0, 0xfc, 0x06, 0x06, 0x06, 0xc6, 0x7c, 0x00, 0x00};
```



Ce genre de représentation / compression s'appelle une **bitmap**. C'est un terme que vous allez grandement retrouver si vous continuez vos recherches dans ce domaine, parce que cela ne concerne pas uniquement le dessin.

Simliste, n'est-ce pas ? Maintenant je vais vous montrer quelques détails qui vont rendre le parsing beaucoup plus casse-pied.

La conversion qu'on vient de faire manuellement implique que chaque caractère aura exactement la même dimension entre eux (8x12 pixels), c'est ce qui s'appelle une "**monospaced font**".

C'est très pratique, mais il faut donc faire attention à votre rendu. Typiquement, la lettre "i" aura possiblement beaucoup d'espace autour de ces lettres voisines, ce qui peut rendre du visuel étrange si la font n'a pas été visuellement travaillée pour.

Pour résoudre les problèmes d'espacement, une nouvelle variante existe : les fonts "proportionnels". C'est globalement la même idée, mais au lieu d'avoir une taille fixe par caractère, vous allez déterminer la surface minimum pour "capturer" le dessin.

L'encodage reste exactement le même nonobstant.

```
// proportional:
// - on ignore les lignes vides
// - on ignore le dernier pixel, car aucune ligne ne l'utilise réellement
// ----- -> /
// #####-- -> 0b1111110x -> /
// ##----- -> 0b11111101,0b100000xx -> 0xfd, /
// ##----- -> 0b10000011,0b00000xxx -> 0x83, /
// #####-- -> 0b00000111,0b1110xxxx -> 0x07, /
// -----##- -> 0b11100000,0b011xxxxx -> 0xe0, /
// -----##- -> 0b01100000,0b11xxxxxx -> 0x60, /
// -----##- -> 0b11000001,0b1xxxxxxx -> 0xc1, /
// ##---##- -> 0b11100011 -> 0xe3
// -#####-- -> 0b0111110x -> / (0x7c)
// ----- -> /
// ----- -> /
u8 const five_bmp[] = {0xfd, 0x83, 0x07, 0xe0, 0x60, 0xc1, 0xe3, ...};
```

Ce qui complexifie grandement le système de rendu et la façon dont on stocke la font final. Typiquement, pour les fonts proportionnels, vous allez devoir stocker, pour chaque caractère, ces informations géométriques afin que, durant le processus de dessin, vous puissiez savoir où sont les données et comment dessiner.

Naïvement, on pourrait imaginer une structure en C qui pourrait ressembler à ça :

```
struct gba_font_monospaced_s {
    u8 bitmap[];
    struct {
        size_t width;
        size_t height;
    } glyph;
};

struct gba_font_proportional_s {
    u8 bitmap[];
    struct {
        size_t width;
        size_t height;
    } glyph[128];
};

typedef struct gba_font_s {
    u8 type; // 0=mono ; 1=proportional
    union {
        struct gba_font_monospaced_s mono;
        struct gba_font_proportional_s prop;
    };
} gba_font_t;
```

Vous avez énormément de stratégies différentes pour gagner en rapidité. Par exemple, vous pourrez ajouter l'index du début des données de dessins de chaque lettre dans `gba_font_proportional_s.glyph[]`, histoire de ne pas avoir à traverser tous les caractères de la terre à chaque fois que l'on veut afficher un caractère.

C'est à vous d'expérimenter et d'optimiser avec ce que vous avez sous la main et vos contraintes (de taille, de temps, ...).

Saisissez-vous du fichier XCF que j'ai dû vous fournir et créez l'outil suivant :

```
> cartridge conv --help
Usage: cartridge conv [OPTIONS] ASSET_PATH

    Convert a font asset into a raw C file

Options:
  -o, --output PATH      generated C file [required]
  -w, --width INT         width size of char in pixel [required]
  -h, --height INT        height size of char in pixel [required]
  -p, --margin INT        margin between char in pixel [required]
  -l, --line-height INT   virtual alignment line height in pixel [required]
  --help                  Show this message and exit.
```



Vous pouvez utiliser n'importe quel format de structure pour stocker vos fonts, la seule contrainte reste la compression en bitmap.

Votre objectif est donc de transformer l'image fournis en fichier C que vous allez devoir compilé est linker avec le reste du programme.



Attention cependant, vous ne supportez pas encore les variables globale, il vous faudra surement mettre a jour votre linker script pour supporter la section `.rodata` (vous ne devez pas implementer le support de `.data`).

Maintenant que nous avons un convertisseur de font, il est temps d'implémenter une interface de dessin pour pouvoir afficher du texte. Vous allez voir que beaucoup de fonctions vont devoir être implémentées. Cependant, vous allez aussi vous rendre compte qu'elles vont toutes tourner autour d'un même cœur.

Voici l'interface à mettre en place:

```
/* Alignment settings for dtext_opt() and dprint_opt(). Combining a vertical
and a horizontal alignment option specifies where a given point (x,y) should
be relative to the rendered string. */
enum {
    /* Horizontal settings: default in dtext() is DTEXT_LEFT */
    DTEXT_VALIGN_LEFT    = 0,
    DTEXT_VALIGN_CENTER  = 1,
    DTEXT_VALIGN_RIGHT   = 2,
    /* Vertical settings: default in dtext() is DTEXT_TOP */
    DTEXT_HALIGN_TOP     = 0,
    DTEXT_HALIGN_MIDDLE  = 1,
    DTEXT_HALIGN_BOTTOM  = 2,
};

/* dtext_opt(): Display a string of text
This function is the core of the text rendering interface */
extern void dtext_opt(
    i32 x, i32 y,           // Coordinates of the anchor of the rendered string
    u16 fg, u16 bg,        // Text color and background color
    u8 halign,             // Where x should be relative to the rendered string
    u8 valign,             // Where y should be relative to the rendered string
    char const *str        // String to display
);

/* dtext(): Simple version of dtext_opt() with defaults
Calls dtext_opt() with bg=C_NONE, halign=DTEXT_LEFT and valign=DTEXT_TOP. */
extern void dtext(i32 x, i32 y, u16 fg, char const *text);

/* dprint_opt(): Display a formatted string
This function is exactly like dtext_opt(), but accepts printf-like formats with
arguments. */
void dprint_opt(
    i32 x, i32 y,           // Coordinates of the anchor of the rendered string
    u16 fg, u16 bg,        // Text color and background color
    u8 halign,             // Where x should be relative to the rendered string
    u8 valign,             // Where y should be relative to the rendered string
    char const *format,    // Printf-like format string to display
    ...                   // Potential variadic arguments
);

/* dprint(): Simple version of dprint_opt() with defaults
Calls dprint_opt() with bg=C_NONE, halign=DTEXT_LEFT and valign=DTEXT_TOP */
extern void dprint(int x, int y, int fg, char const *format, ...);

/* dtext_size(): Get the width and height of rendered text
This function computes the size that the given string would take up if
rendered */
extern void dtext_size(
    i32 *width,            // Rendered width
```

```

    i32 *height,          // Rendered height
    char const *str       // String to display
);

/* dprint_size(): Get the width and height of rendered formatted string
   This function is exactly like dtext_size(), but accepts printf-like formats
   with arguments. */
extern void dprint_size(
    i32 *width,          // Rendered width
    i32 *height,         // Rendered height
    char const *format,  // Printf-like format string to display
    ...                 // Potential variadic arguments
);

```

Une interface de dessins minimaliste en somme.



Pour ce qui est des fonctionnalités des "formatted string", implémenter a minima le `%x`, `%p` et le `%d` si newlib ne vous le permet pas déjà. Vous pouvez implémenter plus si vous le souhaitez, à voir vos besoins.

Premier menu

Maintenant que vous avez implémenté l'API de dessins, il est grand temps d'enfin commencer à afficher des trucs à l'écran. Votre but est simple : vous devez réussir à refaire le screen suivant :

todo: screen

Le screen regroupe donc plusieurs informations :

- ✓ la taille de la ROM
- ✓ la taille du code dans la cartouche
- ✓ la taille de donnée constante (`.rodata`)
- ✓ l'adresse de la stack **une fois la fonction `main()` appelée**
- ✓ le contenu du registre `CPSR`
- ✓ le contenu du registre `IME`

A noter que les registres `CPSR` et `IME` devront être envoyés à la fonction `main()` qui devra ressembler sensiblement à ça:

```
void main(u32 CPSR, u32 IME)
{
    u32 rom_size;
    u32 code_size;
    u32 const_size;
    u32 stack_addr;

    stack_addr = cpu_get_stack();
    // rom_size = ...
    // code_size = ...
    // const_size = ...

    dinit();
    dclear(C_BLACK);
    dtext_opt(
        D_WIDTH/2, 12,
        C_NONE, C_WHITE,
        DTEXT_HALIGN_CENTER, DTEXT_VALIGN_MIDDLE,
        "Welcome!",
    );
    dprint(1, 20, C_BLACK, "ROM: %do", rom_size);
    dprint(1, 32, C_BLACK, "Code: %do", code_size);
    dprint(1, 44, C_BLACK, "read-only data: %do", const_size);
    dprint(1, 56, C_BLACK, "stack: %p", stack_addr);
    dprint(1, 68, C_BLACK, "CPSR: %08x", CPSR);
    dprint(1, 80, C_BLACK, "IME: %08x", IME);

    while(true) { }
```

Ce screen, qui semble assez simpliste, demande quand même une certaine ingénierie derrière. Typiquement, pour récupérer la taille de la ROM, vous allez devoir modifier le linker script afin d'y ajouter de quoi

calculer la taille de certaines sections ; pour la récupération de la stack, vous allez sûrement devoir écrire un bout de code en assembleur pour renvoyer la stack ; pour les registres, vous allez devoir modifier le bootstrap pour envoyer les informations à la fonction `main()` ; ...

Bon courage!

v 0.3.0-rc0

{EPITECH}